Comparative Analysis of Large Language Models in Solidity Smart Contract Vulnerability Detection: Review

Amjad Almaghthawi
Computer Science Department, College of computer Science and Engineering,
Taibah University,
Madinah, Saudi Arabia
tu4570384@taibahu.edu.sa

Wael M.S. Yafooz (IEEE Senior Member)
Computer Science Department, College of computer Science and Engineering,
Taibah University,
Madinah, Saudi Arabia
wyafooz@taibahu.edu.sa

2025 2nd International Conference on Advanced Innovations in Smart Cities (ICAISC) | 979-8-3315-0699-5/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICAISC64594.2025.10959494

***Abstract*— Blockchain (BC) technology and Artificial Intelligence (AI) are promising areas with potential for powerful, secure, and decentralized applications in various sectors. Also, LLMs, are essential advanced machine learning frameworks, which are now used in various applications, including customer service, chatbots, code generation, and language translation. In terms of software development, LLMs are being investigated as a potential way to create smart contracts, focusing on their ability to emulate human programming techniques. For that purpose, this study reviews the capabilities and limitations of Large Language Models (LLMs) like GPT-3.5, PaLM2, and Code Llama in detecting vulnerabilities in Solidity smart contract code.**

Keywords*— *Large Language Models (LLMs), Vulnerability Detection, Smart Contract, GPT-3.5, Code Llama, Palm 2, Solidity, Prompt Engineering

I. Introduction

Researchers are focusing on detecting vulnerabilities in the Solidity smart contracts due to their financial impact, the irreversibility of blockchain transactions, the growing adoption of blockchain technology, the contract complexity, and the constant evolution of attack vectors [1], [2]. To gain a deeper understanding, Figure 1 depicts Ethereum's smart contracts and blockchain interaction shows user-controlled accounts as well as smart contracts. Additionally, it shows the Ethereum blockchain, which includes blocks with hashes, transaction data, and nonces.

To overcome these challenges, traditional and advanced methods are being used. Traditional methods include static analysis and dynamic analysis, combined with machine learning and deep learning, to identify flaws in smart contract code [3]. However, as the complexity of smart contracts increases, more advanced solutions are needed. Large Language Models (LLMs) have emerged as an innovative approach to analyzing and understanding code patterns.

Thus, this study provides an overview of how LLMs can enhance Solidity smart contract vulnerability detection, and it also examines vulnerability detection methods over time. It is important to pay attention to this issue given the increasing importance of smart contracts for blockchain applications. Understanding the strengths and limitations of different LLMs is crucial for practitioners seeking to implement effective security measures into their smart contracts.

Consequently, future work can leverage this foundation to develop more robust solutions by refining prompt designs and exploring ensemble approaches. As a result of this review, we are not only paving the way for enhanced vulnerability detection, but we are also encouraging continued research into optimizing LLMs in real-world blockchain security applications.

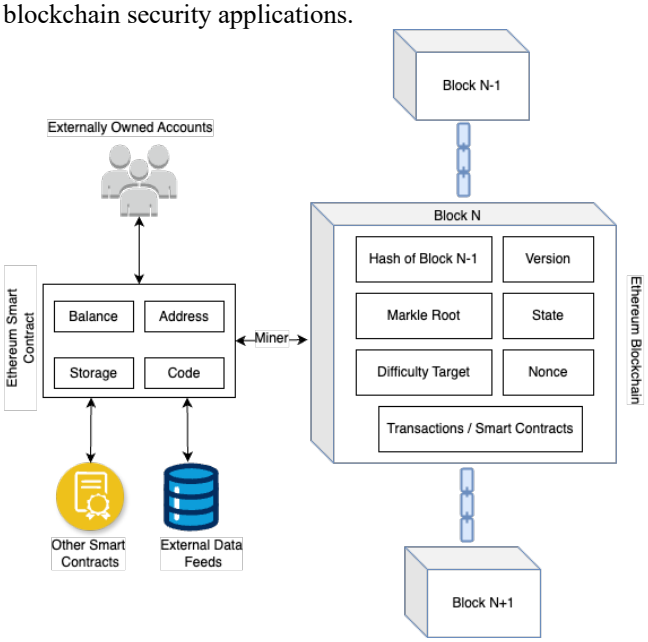


Figure 1: A smart contract based on Ethereum's blockchain [4].

II. Common Vulnerabilities Type

For smart contracts to be more secure and reliable, we must understand their vulnerabilities. However, decentralized applications can suffer significant financial losses and reputation damage due to vulnerabilities [5]. Figure 2 below demonstrates the most common vulnerabilities.



Figure 2:Top 9 vulnerabilities in smart contracts [8].

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 13:23:41 UTC from IEEE Xplore. Restrictions apply.

- Reentrancy

It is a common vulnerability that occurs when a contract invokes an external contract or address, allowing the external contract to make recursive calls back into the calling contract before it has completed its initial function execution [6].

- Arithmetic Issues

Integer Overflow or Underflow is a common type of Arithmetic Issue. It occurs whenever an arithmetic operation exceeds the maximum value of the data type, which causes the value to "wrap around" to the minimum value [7]. Attackers can modify a function's check to increase an Ether user's balance [8], [9].

- Access Control

Strong access control in Solidity smart contracts is essential for asset protection and security. Developers should use modifiers in Solidity to enforce access restrictions, define roles and permissions, regularly audit access control logic, and implement multi-signature wallets for high-value actions [7].

- Bad Randomness

Randomness in Ethereum and Solidity is challenging due to blockchain systems' deterministic nature. It is derived from block-related data, but miners can manipulate it. This can compromise the integrity and fairness of smart contract applications [7].

- Time Manipulation

Smart contracts on Ethereum Virtual Machines (EVMs) rely on block timestamps for execution. Block timestamps are crucial for time sensitive operations, but can be exploited by malicious actors to disrupt contract flow. Also, resulting in delay or accelerate functions, and can lead to financial loss or resource drain [6].

- Short Address Attack

Smart-contract functions encode parameters in 32 bytes, causing a 2-byte left shift after EVM padding. Short Address Attacks exploit third-party applications' handling, leading to incorrect data interpretation. EVM processes transaction data without reverting based on address length [10].

- Front Running

The Ethereum blockchain allows public access to unconfirmed transactions, allowing users to reveal solutions to complex cryptographic puzzles or other sensitive information. An attack vector occurs when a public smart contract publishes an RSA number, and a caller is rewarded for submitting a solution. If someone sends a higher fee, the second transaction is accepted [9].

- Denial of Service (DoS)

DoS attacks can permanently or temporarily disrupt smart contracts, potentially leading to financial losses. To prevent these, developers should avoid unbounded loops, control iterations, and split data processing [7].

- Unchecked Return Values (UR)

The UR value is a ticking time bomb in the execution of smart contracts. When a function returns a value, it indicates whether the operation was successful or failed. In the absence of this value, the contract can proceed as if the operation had succeeded, potentially leading to unexpected and disastrous outcomes [11].

III. Traditional methods

This section discusses two traditional methods for identifying security flaws in Solidity smart contracts: static and dynamic analysis, and machine learning and deep learning.

A. Static and Dynamic Analysis Tools

Static analysis examines code without executing it, focusing on vulnerabilities like reentrancy and access control weaknesses. Tools like Mythril and Slither are used for this purpose, While dynamic analysis tests behavior in a simulated environment such as Echidna and Manticore [4]. Durieux et al. used two datasets to evaluate nine automated analysis tools for Ethereum smart contracts. All tools detected only 42% of vulnerabilities, with Mythril being the most accurate at 27%. In 97% of cases, false positives are identified. As a result, they found certain vulnerabilities could be detected only with limited success [12].

Yashavant et al. demonstrate how tools used to detect vulnerabilities in Ethereum smart contracts lack a standardized dataset for evaluation. An annotated data set called ScrawlD was created by the researchers to serve as an unbiased benchmark for evaluating the effectiveness of existing tools and new ones for finding vulnerabilities in Ethereum smart contracts. As of now, there are 46,000 smart contracts in the system, but the process of labeling them is still ongoing [13].

Furthermore, Qian et al. examine smart contract vulnerability detection techniques. There are three levels of analysis: Solidity code layer, Ethereum virtual machine execution layer, and block dependency layer. According to its review, methods are divided into: formal verification, fuzzy detection, symbolic execution, intermediate representation, and deep learning. They compare accuracy, F1-Score, and average detection time using 300 real-world Ethereum smart contracts. In conclusion, they emphasize the importance of smart contract security and suggest ways to detect vulnerabilities more efficiently [14].

In another research for Choi et al. [15], SMARTIAN implemented to enhance smart contract fuzzing by using static and dynamic data-flow analyses, which enhance vulnerability detection. To determine transaction orders, static analysis preprocessed EVM contracts, while dynamic analysis analyzed feedback to uncover critical paths and discover vulnerabilities. With SMARTIAN, there were 211 more bugs found than with traditional tools.

SWAT (SWC-based Analysis Tool), introduced by Songsom et al. [16], is a static analysis tool that detects Smart Contract Weakness Classification (SWC) vulnerabilities in smart contracts. The tool targets six SWC vulnerabilities using a pattern-matching approach. The SWAT tool is 44.58% more memory-efficient than existing tools like Slither, Mythril, and Vandal in covering SWC vulnerabilities in addition to covering them more efficiently.

Also, Li et al. [17] describe a symbolic execution-based method for detecting vulnerabilities in Ethereum smart contracts, focusing on arbitrary modifications by owners. It addresses problems like limited code coverage and time consumption by exploring all possible execution paths and states. A number of datasets such as the CVE and SB Wild datasets are used, as well as the processing of source code,

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 13:23:41 UTC from IEEE Xplore. Restrictions apply.

the construction of control flow graphs, and the simulation of EVM execution. Compared to existing tools like Mythril and Slither, evaluations indicate high accuracy in detecting owner modification vulnerabilities. Table 1 demonstrates a summary of some papers discussed above.

Table 1: A summary of the static/dynamic analysis tools utilized in previous studies.

[Ref]	Approach/techniques	Data	Size	Vulnerability classes	result
[12]	Analysed 9 tools	Curated, wild	curated 69 wild 47,587	DASP10	Mythril 27%.
[15]	Fuzzing	Ethereum	500	N/A	Found 211 bugs.
[16]	pattern-matching	SWC	N/A	6 SWC	44.58% memory.
[17]	symbolic execution	CVE, SB Wild	CVE 13, SB Wild 5,000	owner arbitrary modification	92.3%

B. Machine Learning (ML) and Deep Learning (DL) Technique

ML involves training and inference algorithms in many domains [18][19][20]. During training, model parameters are optimized, while inference uses unlearned data to infer features. In DL, features are extracted using neural networks in a black-box manner, which is most popular today [21]. Research in this area is very active, Wang et al. developed ContractWard which is a ML-based approach for efficient accuracy of 98.47% in detecting these vulnerabilities, showing superior performance compared to previous models.

Furthermore, Boxin [26] presents a dynamic vulnerability detection approach to smart contracts. Seeks to identify vulnerabilities within deployed smart contracts by analyzing opcode sequences during their dynamic execution. The approach uses the N-gram model and a weight penalty mechanism for feature extraction, and uses models such as KNN, SVM, and LR for training and detecting vulnerabilities. The dataset includes two main categories: normal samples and vulnerability samples. Yet, the approach demonstrated the effectiveness of SVM with its 93.71% detection accuracy and 92.55% F1 score. In Yang et al. [27], a ML approach is presented for automating vulnerability detection in smart contracts. Text Convolutional Neural Networks (TextCNN) and an attention mechanism are used to learn contextual relationships within the code. Compared to baseline models, the model is effective at improving blockchain security.

Yang et al. present method based on an Abstract Syntax Tree (AST) for detecting vulnerabilities in smart contracts. In this method, AST-based representations are used to automatically learn and detect vulnerabilities, resulting in high accuracy and effectiveness [28].

Moreover, based on Graph Neural Networks (GNN) and Convolutional Neural Networks (CNN), Han et al. [29] present a ML model for detecting vulnerabilities in Ethereum smart contracts. In order to improve detection accuracy, the model uses control flow graphs and GNNs to learn semantic and structural features associated with vulnerabilities.

and accurate vulnerability detection in Ethereum smart contracts proposed. The system uses a large dataset of 49,502 contracts and employs multiple algorithms to build detection models. ContractWard significantly improves detection efficiency and accuracy, with 96% predictive scores. It can detect vulnerabilities in 4 seconds per contract, reducing detection time and maintaining high precision [22].

In another study by F.Luo and R.Luo, VulDet, a novel vulnerability detection method for smart contracts using Graph Attention Networks. It focuses on detecting reentrancy and timestamp dependency vulnerabilities in smart contract cod. To train and test the Ethereum Smart Contract dataset, 40,932 contracts were used. Using a GAT approach, VulDet constructs contract graphs, uses domain-specific features, and focuses on critical aspects. When compared to Vanilla-RNN, LSTM, and DR-GCN deep learning algorithms, VulDet outperforms them all [23].

In addition, according to Liu et al. [24], the Transfer Learning-Based Smart Contract Generation (TLSCG) framework can improve smart contract vulnerability detection, particularly for unknown vulnerabilities. This framework uses two datasets: the Ethereum Dataset and the SBcurated Dataset. In terms of anomalous contract detection, TLSCG is 72% accurate, outperforming existing tools by up to 18%. Wang et al. [25] aim to enhance smart contract vulnerability detection using machine learning by developing a model that covers a broad range of vulnerability types. They build a dataset named SC-4 includes over 3000 data points for four common vulnerabilities: Reentrancy, Transaction Order Dependence, Unchecked-Send, and Unhandled-Exceptions. The proposed GRU-RF algorithm achieved

For detecting reentrancy vulnerabilities in Solidity smart contracts, Demir et al. present a deep learning-based framework. To detect vulnerabilities rapidly and accurately, the system combines Op2Vec vectorization with a Bi-LSTM model. In testing, the model identified reentrancy vulnerabilities with 96% accuracy, notably reducing analysis time compared to static analyzers [30]. A summary of the studies is presented in Table 2.

IV. ADVANCED METHODS

This section illustrates a general overview of the three most commonly used LLMs, namely GPT-3, PaLM2, and Code Llama. However, all studies discussed in this section summarized in tables 3 and 4.

A. GPT-3.5

OpenAI's GPT3.5 and GPT-3 language models, trained using publicly available data and Reinforcement Learning from Human Feedback, provide fluency, coherence, and contextual understanding for various applications [31]. Hu et al. examines smart contract vulnerabilities using LLMs, specifically GPT-4. In order to detect vulnerabilities in smart contracts, the paper proposes an adversarial framework referred to as GPTLENS that takes advantage of the LLM's dual roles as auditor and critic. However, by balancing the generation of vulnerabilities and minimizing false positives, the GPTLENS framework improves smart contract auditing significantly without requiring specialized expertise in smart

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 13:23:41 UTC from IEEE Xplore. Restrictions apply.

Table 2: A summary previous studies that use the ML/DL in detection

[Ref]	Approach/ techniques	Data	Size	Vulnerability classes	result
[22]	XGBoost, AdaBoost, RF, SVM, and KNN.	Ethereum	49,502	6	96%
[23]	Graph Construction, GAT	ESC	40,932	2	Reentrancy 4.66%, timestamp-dependency 6.05%.
[24]	Transfer Learning, Anomaly Detection	Ethereum, SBcurated	Ethereum 30,137, SBcurated 78	4	72%
[25]	GRU and RF	SC-4	3000 per Vuln.	4	98.47%
[26]	KNN, SVM, and LR	opcode sequences	3,685,499	6	SVM 93.71%, 92.55% F1.
[27]	TextCNN, Attention Mechanism	SmartBugs	3735	5	99.20%
[28]	RF, Word2Vec	EtherScan	3000	3	98.7% and 0.885 F1.
[29]	GNNs, CNNs	Eth2Vec	5000	4	94.33% and 92.60%F1.
[30]	Op2Vec, Bi-LSTM	online sources	30,000	1	96%

contracts. It detected vulnerabilities in 76.9% of contracts, while traditional methods achieved only 38.5% [32].

Also, Chen et al. examines ChatGPT ability to identify smart contract vulnerabilities. Moreover, they are comparing it with other tools. It evaluates precision and recall. Researchers analyze their findings to gain insight into the advantages, disadvantages, and limitations of using large language models (LLM) to detect smart contract vulnerabilities. However, ChatGPT showed promise in detecting some smart contract vulnerabilities, but it struggled with precision and displayed false positives. In comparison to 11 other tools, it detected 77.8% of vulnerability types poorly [33].

Furthermore, Gao et al. [34] study ChatGPT's capabilities of identifying Machine Unauditable Bug (MUBs) within smart contracts. Smart contracts typically contain 80% of exploitable bugs, making identifying these vulnerabilities crucial to enhancing blockchain security. ChatGPT's performance is compared with SPCON, a state-of-the-art tool for detecting access control bugs. Using category-specific prompts, ChatGPT detected 33% of high-risk MUBs when guided by different categories of MUBs. While ChatGPT struggles with specific implementation bugs, its effectiveness is greatly affected by prompt structure and bug nature. Compared to SPCON, ChatGPT identifies 9 out of 17 vulnerabilities.

Boi et al. [9] developed VulnHunt-GPT to detect vulnerabilities in smart contracts using OpenAI's GPT-3 model. Using natural language processing (NLP), the model analyzes code, identifies vulnerabilities, and suggests remediations based on the OWASP Top 10 categories. Testing was conducted on a curated vulnerability dataset, and it achieved broad coverage. Thus, based on an average execution time of 23 seconds per contract, VulnHunt-GPT successfully detected 128 vulnerabilities in smart contracts.

A study by Du et al. [35] evaluates GPT-4's performance in auditing smart contracts for vulnerabilities. The code parsing abilities of the program were tested using datasets from SolidiFI-benchmark library and eight sets of smart contracts. However, GPT-4's code parsing abilities were examined using Chain of Thought (CoT) prompts. GPT-4 detects complex smart contract vulnerabilities with high precision, but low recall. Furthermore, the tool showed mixed results when generating Proof of Concept (PoC) exploits.

Table 3: A summary of previous studies that use GPT-3.5 in detection

[Ref]	Approach/ techniques	Data	Size	Vulnerability classes	result
[32]	Open-ended prompting	Etherscan	13	CVEs	76.9%
[33]	Compare ChatGPT's with 14 tools	Smartbugs-curated	142	DASP10	GPT-4 88.2% recall, 22.6% precision.
[34]	General prompts, guidance prompts	Code4rena CVE	246, 17 CVE	6 MUBs	33%
[9]	prompt engineering, context enrichment	Smartbugs curated	69	OWASP	128 detected
[35]	CoT prompting	SolidiFI-benchmark	35	9	33%, 80% precision, 11% recall.

B. PaLM 2

PaLM2, a multilingual system, powers AI chatbots, writes code, analyzes images, and translates, with advanced language proficiency and can process mathematical expressions and scientific journals [36]. Ullah et al. investigate the effectiveness of large language models (LLMs) for detecting code vulnerabilities. LLMs are assessed using SecLLMHolmes, a framework that assesses their reasoning capacity and accuracy. Based on a dataset of 228 code scenarios, eight models are tested against various vulnerabilities and complexities. LLMs struggle with consistent and accurate vulnerability detection, producing non-deterministic outputs and flawed reasoning in the results. In terms of PaLM 2, it has potential, but requires further advancements to be a reliable tool for automated security vulnerability detection [37].

C. Code Llama

The Code Llama LLM family is optimized for code generation and infilling. There are three versions of Code Llama: General, Python, and Instruction. The models have been trained extensively and demonstrate competitive performance [38]. The SmartVD vulnerability detection framework was developed by Alam et al. [39] to detect

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 13:23:41 UTC from IEEE Xplore. Restrictions apply.

vulnerabilities in smart contracts written in Solidity using LLMs. Binary classification and multi-class classification are used to identify specific types and severity of vulnerabilities. It utilizes a novel dataset, VulSmart, comprised of 6,085 smart contracts. GPT-3.5 and CodeLlama achieve the best scores when fine-tuning is applied. Despite improvements in precision, recall, and F1-score, CodeLlama still struggles in handling complex adversarial cases.

Also, Ince et al. examines the effectiveness of Meta's Code Llama models for identifying vulnerabilities in smart contracts. The authors propose that these models, specifically Detect Llama - Foundation and Detect Llama - Instruct, can outperform OpenAI's GPT-4. The best-performing model, GPT-3.5 Turbo Fine-tuned, achieves higher weighted F1 scores than GPT-4 and GPT-4 Turbo [40].

Table 4: A summary of previous studies that uses PaLM 2/ Code Llama LLMs in detection.

[Ref]	Approach/ techniques	Data	Size	Vulnerability classes	result
[37]	Evaluate 8 LLMs, Diverse prompting	CWE, CVEs, Code Augmentations	228	8 CWEs	PaLM2 26% incorrect, GPT-4 17% incorrect.
[39]	zero-shot	VulSmart	6,085	OWASP 10	99% detection, 94% type identification, and 98% severity.
[40]	Random Baseline	ScrawlD	17,000 prompts, 4,000 prompts	8	0.61 F1

V. Conclusion

A growing number of researchers are interested in detecting vulnerabilities in Solidity smart contracts because of their financial impact, irreversibility of the blockchain transactions, and complexity of the contracts. Traditional methods such as static analysis, ML and DL have been employed. In the meantime, LLMs have emerged as an innovative approach to analyzing and understanding code patterns. However, this chapter presents an overview of common vulnerabilities including reentrancy, arithmetic issues, access control vulnerabilities, bad randomness, time manipulation, short address attacks, front running, DoS, and UR. Along with exploring some existing datasets as well as discussing studies of vulnerability detection using traditional and advanced methods, focusing on three LLMs, GPT-3.5, Code Llama, and PaLM2

References

[1] H. Li, R. Dang, Y. Yao, and H. Wang, "A Review of Approaches for Detecting Vulnerabilities in Smart Contracts within Web 3.0 Applications," *Blockchains*, vol. 1, no. 1, Art. no. 1, Sep. 2023, doi: 10.3390/blockchains1010002.

[2] M. Xiao, Y. XU, Z. LI, and H. Wan, "Advanced Security Auditing Methods for Solidity-Based Smart Contracts - ProQuest." Accessed: Jan. 03, 2025. [Online]. Available: https://www.proquest.com/openview/b561812ce0b363f1735b4afa71d8fa6b/1?pq-origsite=gscholar&cbl=2032404

[3] F. Vidal, N. Ivaki, and N. Laranjeiro, "Vulnerability detection techniques for smart contracts: A systematic literature review," *2024*, Nov. 2024.

[4] S. S. Kushwaha, S. Joshi, D. Singh, M. Kaur, and H.-N. Lee, "Ethereum Smart Contract Analysis Tools: A Systematic Review," *IEEE Access*, vol. 10, pp. 57037–57062, 2022, doi: 10.1109/ACCESS.2022.3169902.

[5] D. He, Z. Deng, Y. Zhang, S. Chan, Y. Cheng, and N. Guizani, "Smart Contract Vulnerability Analysis and Security Audit," *IEEE Netw.*, vol. 34, no. 5, pp. 276–282, Sep. 2020, doi: 10.1109/MNET.001.1900656.

[6] P. Tantikul and S. Ngamsuriyaroj, "Exploring Vulnerabilities in Solidity Smart Contract:," in *Proceedings of the 6th International Conference on Information Systems Security and Privacy*, Valletta, Malta: SCITEPRESS - Science and Technology Publications, 2020, pp. 317–324. doi: 10.5220/0008909803170324.

[7] M. Ortu, G. Ibba, G. Destefanis, C. Conversano, and R. Tonelli, "Taxonomic insights into ethereum smart contracts by linking application categories to security vulnerabilities," *Sci. Rep.*, vol. 14, no. 1, p. 23433, Oct. 2024, doi: 10.1038/s41598-024-73454-0.

[8] Z. Khan and A. Namin, "Ethereum Smart Contracts: Vulnerabilities and their Classifications." Accessed: Oct. 28, 2024. [Online]. Available: https://www.computer.org/csdl/proceedings-article/big-data/2020/09439088/1tRPhOx5Gko

[9] B. Boi, C. Esposito, and S. Lee, "VulnHunt-GPT: a Smart Contract vulnerabilities detector based on OpenAI chatGPT," in *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing*, Avila Spain: ACM, Apr. 2024, pp. 1517–1524. doi: 10.1145/3605098.3636003.

[10] H. Zhou, A. Milani Fard, and A. Makanju, "The State of Ethereum Smart Contracts Security: Vulnerabilities, Countermeasures, and Tool Support," *J. Cybersecurity Priv.*, vol. 2, no. 2, Art. no. 2, Jun. 2022, doi: 10.3390/jcp2020019.

[11] D. Perez and B. Livshits, "Smart Contract Vulnerabilities: Vulnerable Does Not Imply Exploited," presented at the 30th USENIX Security Symposium (USENIX Security 21), 2021, pp. 1325–1341. Accessed: Oct. 29, 2024. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/perez

[12] T. Durieux, J. F. Ferreira, R. Abreu, and P. Cruz, "Empirical review of automated analysis tools on 47,587 Ethereum smart contracts," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, in ICSE '20. New York, NY, USA: Association for Computing Machinery, Oct. 2020, pp. 530–541. doi: 10.1145/3377811.3380364.

[13] C. S. Yashavant, S. Kumar, and A. Karkare, "ScrawlD: A Dataset of Real World Ethereum Smart Contracts Labelled with Vulnerabilities," Feb. 25, 2022, *arXiv*: arXiv:2202.11409. Accessed: Mar. 12, 2024. [Online]. Available: http://arxiv.org/abs/2202.11409

[14] P. Qian, Z. Liu, Q. He, B. Huang, D. Tian, and X. Wang, "Smart Contract Vulnerability Detection Technique: A Survey," Sep. 13, 2022. doi: 10.13328/j.cnki.jos.006375.

[15] J. Choi, D. Kim, S. Kim, G. Grieco, A. Groce, and S. K. Cha, "SMARTIAN: Enhancing Smart Contract Fuzzing with Static and Dynamic Data-Flow Analyses," in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, Nov. 2021, pp. 227–239. doi: 10.1109/ASE51524.2021.9678888.

[16] N. Songsom, W. Werapun, J. Suaboot, and N. Rattanavipanon, "The SWC-based Security Analysis Tool for Smart Contract Vulnerability Detection," in *2022 6th International Conference on Information Technology (InCIT)*, Nov. 2022, pp. 74–77. doi: 10.1109/InCIT56086.2022.10067759.

[17] X. Li, J. Liu, X. Chen, and Q. Zhang, "A Symbolic Execution-Based Approach for Smart Contract Vulnerability Detection | IEEE Conference Publication | IEEE Xplore." Accessed: Nov. 13, 2024. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10407615

[18] W. M. S. Yafooz, "Enhancing Arabic Dialect Detection on Social Media: A Hybrid Model with an Attention Mechanism," *Information*, vol. 15, no. 6, Art. no. 6, Jun. 2024, doi: 10.3390/info15060316.

[19] S. Mandala, A. I. Ramadhan, M. Rosalinda, W. M. S. Yafooz, and R. H. Khohar, "DDoS Detection by Using Information Gain-Naïve Bayes," in *2022 2nd International Conference on Intelligent Cybernetics Technology & Applications (ICICyTA)*, Dec. 2022, pp. 283–288. doi: 10.1109/ICICyTA57421.2022.10038054.

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 13:23:41 UTC from IEEE Xplore. Restrictions apply.

[20] R. F. Alhujaili and W. M. S. Yafooz, “Sentiment Analysis for YouTube Educational Videos Using Machine and Deep Learning Approaches,” in *2022 IEEE 2nd International Conference on Electronic Technology, Communication and Information (ICETCI)*, May 2022, pp. 238–244. doi: 10.1109/ICETCI55101.2022.9832284.

[21] N. Ashizawa, N. Yanai, J. P. Cruz, and S. Okamura, “Eth2Vec: Learning Contract-Wide Code Representations for Vulnerability Detection on Ethereum Smart Contracts,” Jan. 01, 2021. doi: 10.48550/arXiv.2101.02377.

[22] W. Wang, J. Song, G. Xu, Y. Li, H. Wang, and C. Su, “ContractWard: Automated Vulnerability Detection Models for Ethereum Smart Contracts,” *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 1133–1144, Apr. 2021, doi: 10.1109/TNSE.2020.2968505.

[23] F. Luo and R. Luo, “VulDet: Smart Contract Vulnerability Detection Based on Graph Attention Networks,” presented at the 2023 2nd International Conference on Artificial Intelligence and Blockchain Technology (AIBT), IEEE Computer Society, Jun. 2023, pp. 88–92. doi: 10.1109/AIBT57480.2023.00024.

[24] C. Liu, Z. Sang, L. Duan, W. Ni, W. Wang, and C. Li, “TLSCG: Transfer Learning-Based Efficient Anomalous Smart Contract Generation to Empower Unknown Vulnerability Detection,” in *2024 IEEE International Conference on Web Services (ICWS)*, Jul. 2024, pp. 468–478. doi: 10.1109/ICWS62655.2024.00068.

[25] D. Wang, M. Shan, and N. Tong, “Smart Contract Vulnerability Detection Based on Machine Learning | IEEE Conference Publication | IEEE Xplore.” Accessed: Nov. 07, 2024. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10696331

[26] Y. Boxin, “Research on Dynamic Detection of Vulnerabilities in Smart Contracts Based on Machine Learning,” in *2024 IEEE 3rd International Conference on Electrical Engineering, Big Data and Algorithms (EEBDA)*, Feb. 2024, pp. 219–223. doi: 10.1109/EEBDA60612.2024.10485831.

[27] Z. Yang, C. Zhou, Q. Xiang, W. Huang, and X. Wang, “Smart Contract Vulnerability Detection Based on TextCNN and Attention Mechanism | IEEE Conference Publication | IEEE Xplore.” Accessed: Nov. 07, 2024. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10483421

[28] H. Yang, J. Zhang, X. Gu, and Z. Cui, “Smart Contract Vulnerability Detection based on Abstract Syntax Tree,” in *2022 8th International Symposium on System Security, Safety, and Reliability (ISSSR)*, Oct. 2022, pp. 169–170. doi: 10.1109/ISSSR56778.2022.00032.

[29] D. Han, Q. Li, L. Zhang, and T. Xu, “A smart contract vulnerability detection model based on graph neural networks,” in *2022 4th International Conference on Frontiers Technology of Information and Computer (ICFTIC)*, Dec. 2022, pp. 834–837. doi: 10.1109/ICFTIC57696.2022.10075325.

[30] H. O. Demir, S. Z. Parlat, and A. Gumus, “Ethereum Blockchain Smart Contract Vulnerability Detection Using Deep Learning | IEEE Conference Publication | IEEE Xplore.” Accessed: Nov. 13, 2024. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10391797

[31] A. Koubaa, “GPT-4 vs. GPT-3.5: A Concise Showdown,” Mar. 24, 2023, *Preprints*: 2023030422. doi: 10.20944/preprints202303.0422.v1.

[32] S. Hu, T. Huang, F. İlhan, S. F. Tekin, and L. Liu, “Large Language Model-Powered Smart Contract Vulnerability Detection: New Perspectives,” Oct. 16, 2023, *arXiv*: arXiv:2310.01152. Accessed: Mar. 07, 2024. [Online]. Available: http://arxiv.org/abs/2310.01152

[33] C. Chen *et al.*, “When ChatGPT Meets Smart Contract Vulnerability Detection: How Far Are We?,” Sep. 14, 2023, *arXiv*: arXiv:2309.05520. Accessed: Mar. 07, 2024. [Online]. Available: http://arxiv.org/abs/2309.05520

[34] B. Gao, Q. Wei, Y. Liu, and R. S. M. Goh, “Unveiling the Potential of ChatGPT in Detecting Machine Unauditable Bugs in Smart Contracts: A Preliminary Evaluation and Categorization | IEEE Conference Publication | IEEE Xplore.” Accessed: Nov. 14, 2024. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10605444

[35] Y. Du and X. Tang, “Evaluation of ChatGPT’s Smart Contract Auditing Capabilities Based on Chain of Thought,” Feb. 19, 2024, *arXiv*: arXiv:2402.12023. doi: 10.48550/arXiv.2402.12023.

[36] R. Bansal *et al.*, “LLM Augmented LLMs: Expanding Capabilities through Composition,” Jan. 04, 2024, *arXiv*: arXiv:2401.02412. doi: 10.48550/arXiv.2401.02412.

[37] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini, “LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet?): A Comprehensive Evaluation, Framework, and Benchmarks for S&P 2024 - IBM Research.” Accessed: Nov. 14, 2024. [Online]. Available: https://research.ibm.com/publications/llms-cannot-reliably-identify-and-reason-about-security-vulnerabilities-yet-a-comprehensive-evaluation-framework-and-benchmarks

[38] B. Rozière *et al.*, “Code Llama: Open Foundation Models for Code,” Jan. 31, 2024, *arXiv*: arXiv:2308.12950. doi: 10.48550/arXiv.2308.12950.

[39] M. T. Alam, R. Halder, and A. Maiti, “Detection Made Easy: Potentials of Large Language Models for Solidity Vulnerabilities,” Sep. 15, 2024, *arXiv*: arXiv:2409.10574. doi: 10.48550/arXiv.2409.10574.

[40] P. Ince, X. Luo, J. Yu, J. K. Liu, and X. Du, “Detect Llama -- Finding Vulnerabilities in Smart Contracts using Large Language Models,” vol. 14897, 2024, pp. 424–443. doi: 10.1007/978-981-97-5101-3_23.

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 13:23:41 UTC from IEEE Xplore. Restrictions apply.